Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

Jun 10, 2020 · 8 min read · ✨ · Listen

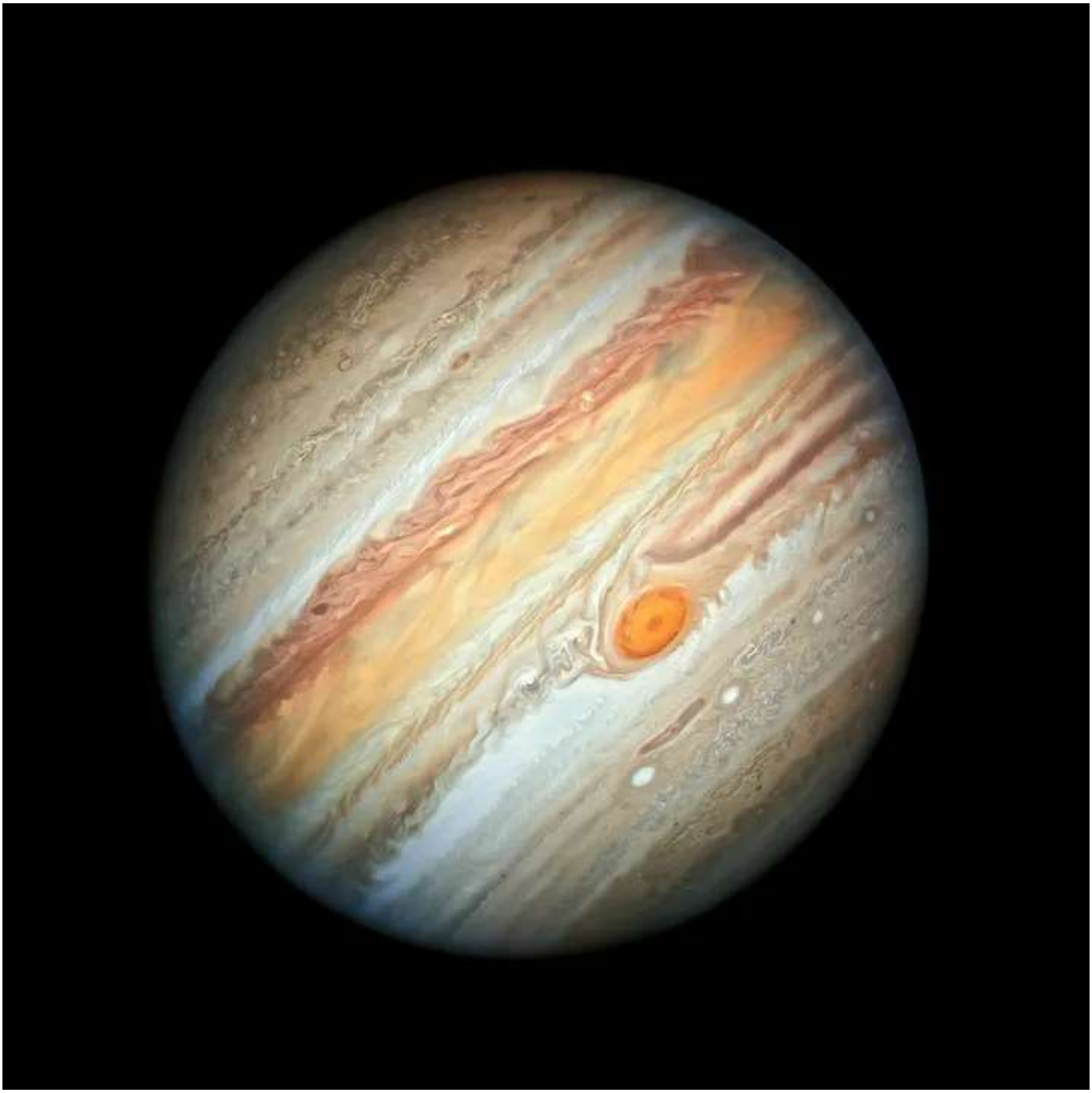
Save

SPACE SCIENCE WITH PYTHON

Space Science with Python — An Invisible Visitor

Part 14 of the tutorial series takes a look at the comet 67P/Churyumov–Gerasimenko after the spacecraft Rosetta and Philae revealed its cosmic mysteries.

2



Has a major influence on smaller bodies in the Solar System: Jupiter. Credit: NASA, ESA, A. Simon (Goddard Space Flight Center), and M.H. Wong (University of California, Berkeley); License: CC BY 4.0

Preface

This is the 14th part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!

Introduction

Last time (Turbulent times of a comet), we took a look at the trajectory (*path*) of the comet 67P/Churyumov-Gerasimenko. We found out that this comet has a turbulent

journey around our central star. The orbital elements change all the time significantly making it difficult to describe it by simple, static orbital elements. For example, the argument of perihelion may change within a few years by several degrees. In comparison: Earth's argument of perihelion *rotates* around the Sun in a time period of 100,000s of years!

But why? Well, it is the planet's fault! Their gravitational pull causes permanent changes in the comet's orbit. And as described previously (A Rendezvous with Jupiter) Jupiter is the main influencer in our cosmic neighbourhood. 67P's past was violently disturbed as numerical simulations by Maquet (2015) [1] have shown.

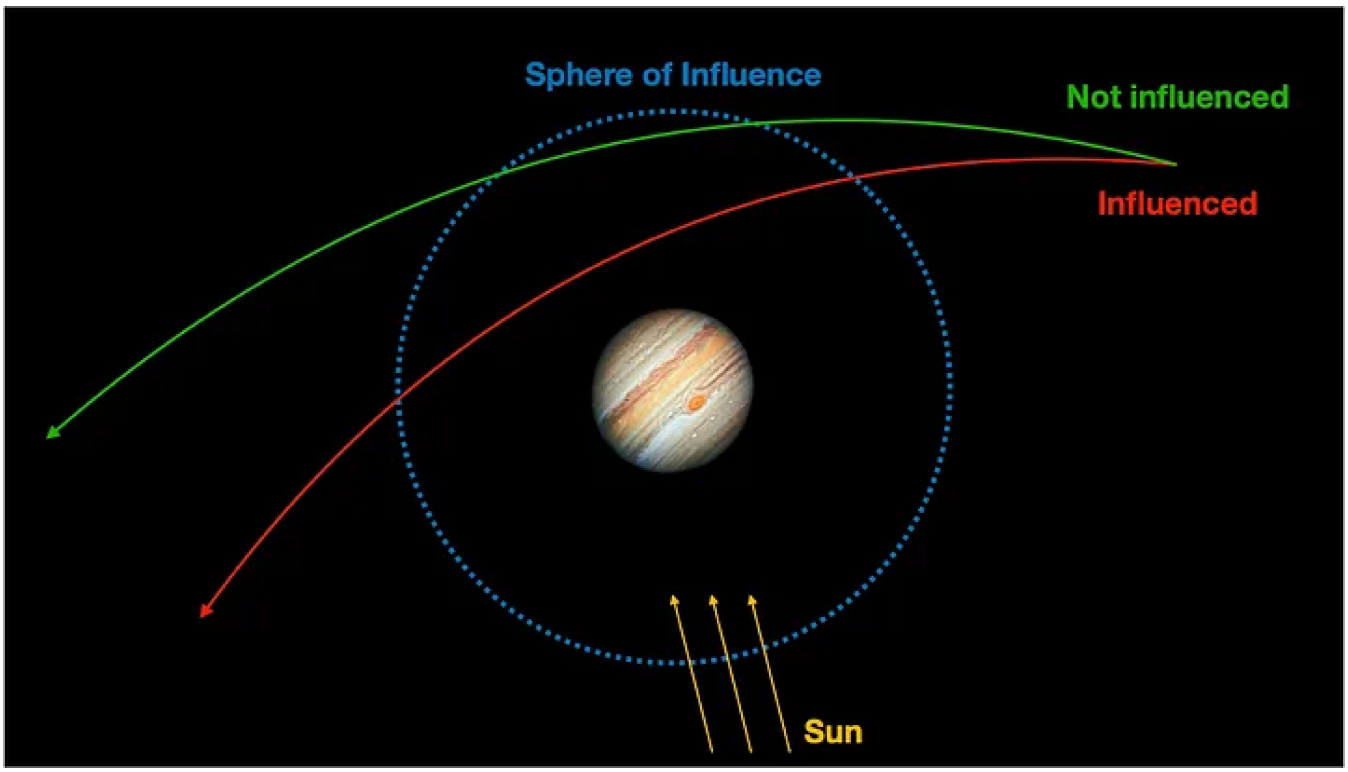
But what about the future, or recent times? Do we need to set up a complex numerical program to get an idea of orbital changes? Luckily, no, we do not need any GPU optimized C code to get an idea. What we need is Python, NASA's toolkit SPICE (respectively the Python Wrapper *spiceypy*) and the awareness that we are trying to get simplified solutions! The final solutions may differ from the *actual* data.

So let's start our coding part. This time, we will merge coding and theory to get a better understanding of the computational and scientific procedure.

In the vicinity of Jupiter

In part 6 of the tutorial series (Around the Sun) I introduced an astro-dynamical concept called: The Sphere-of-Influence (SOI). Each celestial body in the Solar System has this imaginary sphere that allows us to reduce N-body problems to 2-body problems in the following way:

An asteroid or comet revolves the Sun with certain orbital elements. In this Sun-centric coordinate system (the mass of the smaller body can be neglected) no other bodies are present that may cause any gravitational perturbation. However, if the asteroid or comet enters the SOI of a planet, the coordinate system is *switched* to a planet-centric coordinate system. The planet is the only reference point and dominant gravity source. If the body leaves the planet's SOI, we switch our coordinate system back to a Sun-centric one.



Sketch of Jupiter's Sphere Of Influence (blue dashed circle). The Sun is located at the bottom, indicated with three yellow sun-rays. A small body, like a comet, revolves the Sun undisturbed in a 2 body problem (green path). However, if one considers the gravitational pull of Jupiter within its SOI, the path changes accordingly (red path), resulting in a changed orbit. Sketch not in scale. Credit: T. Albin, Credit of Jupiter's image: Credit: NASA, ESA, A. Simon (Goddard Space Flight Center), and M.H. Wong (University of California, Berkeley); License of Jupiter's image: CC BY 4.0

Now, this method may appear questionable, since the gravity of a body has no spatial limitations. Anyway, this simplification will help us to derive some interesting orbit changing insights.

We need our modules that we used now several times. The kernel meta file contains miscellaneous relative paths to the necessary kernel files that are also uploaded on my GitHub repository. Besides the standard leap-seconds kernel (*naif0012.tls*) and spk kernels for our planets (*de432s.bsp*) we will also use an spk kernel of 67P to derive orbital elements. The corresponding kernel *67P_CHURY_GERAS_2004_2016.BSP* contains positional information of the comet between the years 2004 and 2016.

Part 1/14

Our goal: we want to determine if and when 67P crosses the SOI of Jupiter. For this purpose, we need to compute state vectors of Jupiter and 67P and we need to determine the SOI's radius! For the SOI, we need the mass of the Sun and Jupiter or the G*M (G: gravitational constant, M: mass) values, respectively. With the function *bodvcd* we extract the G*M value for the Sun (line 2 an 3) and Jupiter (line 6 and 7). The NAIF IDs for both objects are 10 and 5:

Part 2/14

Now we set a sample Ephemeris time (ET) in line 3 and compute the state vector of Jupiter in ECLIPJ2000 as seen from the Sun (line 6 to 9) with the function *spkgeo*. For

the SOI computation, we need the semi-major axis of Jupiter. Orbital elements can easily be computed with *oscltx* as shown in line 12 to 14. The resulting array *JUPITER_ORB_ELEM* contains the orbital elements of Jupiter at the *sample_et* time and the second to last entry contains the semi-major axis (line 17). Let's print the result in AU to see whether the extracted value appears to be feasible (using the function *convrt*):

Part 3/14

A semi-major axis of around 5.2 AU looks good! So let's continue with the computation of …

```
Semi-major axis of Jupiter in AU: 5.2097194462305065
```

... the SOI. Line 2 defines the radius using the already extracted parameters (semi-major axis, G*M value of the Sun and Jupiter). The resulting radius is ...

Part 4/14

... around 0.3 AU! That is around 50 Million kilometres, respectively one-third of the distance between our home planet and the Sun! Jupiter's influence is indeed immense.

```
SOI of Jupiter in AU: 0.32268429545253885
```

Our loaded 67P SPICE kernel is only valid for a brief period of time. To extend the orbit prediction range, we will use the function *conics* to compute 67P's state vectors based on its orbital elements. To obtain the orbital elements of 67P, we set a sample ET (line 6) and compute the corresponding state vector in line 9 to 12. Then, we use *oscelt* to determine the orbital elements (line 15 to 17).

Part 5/14

We are now set to determine the intersection between 67P and Jupiter's SOI. We start at the first January 2017 (line 3) and create a while condition (line 12 to 29) that

does the following:

- The while condition checks after each iteration if the distance between 67P and Jupiter is larger than the radius of the SOI (line 12).
- If 67P is still outside Jupiter's SOI, the date-time is increases by 1 hour (line 15) and converted to ET (16).
- The ET corresponding state vector of 67P is computed (line 20) with the function *conics*. The resulting state vector is Sun-centric.
- Additionally, the state vector of Jupiter is computed (line 23 to 26); also in a Sun-centric ECLIPJ2000 system.
- Finally, the distance between the comet and the planet is computed in line 29 using the function *vnorm*.

Part 6/14

We execute the code and after a very short period of time the while condition is not fulfilled anymore. We reached the first SOI intersection with Jupiter after the 1st January 2017. Let's take a look at the corresponding date (line 4). Further, we compute also the distance between both objects to verify whether our code worked properly (line 5 and 6; we convert the value from km to AU with the function *convrt*).

Part 7/14

It appears that 67P entered Jupiter's SOI 2 years ago!

```
67P entering Jupiter's SOI: 2018-05-20
67P distance to Jupiter at SOI crossing in AU: 0.3226421083848195
```

Did 67P's journey within the SOI have any influence on its trajectory? Did the orbital elements in ECLIPJ2000 really changed? Let's find out!

We need to compute the movement of 67P in a Jupiter-centric system. For this purpose, we compute the state vector of the comet as seen from Jupiter, by a simple vector calculation (line 3). The resulting vector is then used to compute the orbital elements of 67P within Jupiter's SOI (line 7 to 9). Please note that we use the G*M parameter of Jupiter (line 9) this time!

Part 8/14

Let's take a look at some results of the Jupiter-centric orbital elements:

Part 9/14

The perijove (so the periapsis with respect to Jupiter), corresponds with the closest distance between 67P and the gas giant. It is around 41.5 Million kilometres which correspond to 86 % of the SOI's radius. Unsurprisingly, the eccentricity of 67P is way beyond 1, indicating that the comet performs an unbound fly-by.

```
Closest distance between 67P and Jupiter in km: 41512816.418108694
Closest distance between 67P and Jupiter in SOI radius percentage:
```

```
86.0
67P's eccentricity in a Jupiter-centric system: 8.642146316093086
```

Now let's compute the trajectory within the SOI until 67P reaches again the border of the sphere. The while loop is similar as shown before, only the while condition has been changed accordingly:

Part 10/14

After some time the program exits the while loop. Let's print the date of the second SOI crossing:

Part 11/14

67P was almost 4 months in the vicinity of Jupiter!

```
67P leaving Jupiter's SOI: 2018-09-09
```

Line 4 and 5 compute now the SOI leaving time corresponding state vector of Jupiter as seen from the Sun. Then, we can re-transform 67P's state vector from the Jupiter-centric to the Sun-centric system.

Part 12/14

Based on the state vector after Jupiter's gravitational influence, we can compute the corresponding orbital elements …

Part 13/14

… and compare the resulting elements with the orbital elements before 67P entered Jupiter's SOI:

As you can see, within this 4 month period where 67P was within Jupiter's SOI, the orbital elements changed significantly. The perihelion decreased by 0.07 AU and the orbit became a little bit more elliptic. The inclination decreased by 1.5 degrees and the longitude of the ascending node, as well as the argument of perihelion shifted by a few tens of arcminutes.

```
Perihelion in AU before: 1.29, after: 1.22
Eccentricity before: 0.6318, after: 0.6463
Inclination in degrees before: 7.12, after: 5.52
Longitude of ascending node in degrees before: 50.93, after: 49.88
Argument of perihelion in degrees before: 11.41, after: 9.28
```

All in all, our short non-numerical SPICE-based code allowed us to get a feeling for the changes of the orbital elements after close encounters. However, there are some limits one has to consider as explained in the conclusion.

Conclusion

No GPU? No problem. *No numerical computational heavy code?* No problem. I have shown you that a simple, well thought 2 body problem approach is sufficient to derive changing orbital elements after close encounters.

However, try some other initial date-times in coding part 5/14. Compute the state vector of 67P for a different time with slightly different orbital elements. You will see that the comet does not enter Jupiter's SOI at all! Small initial changes in the orbital elements cause a *Butterfly-Effect*: small changes cause vast changes in the future. Predicting highly accurate orbits requires numerical simulations, the outgassing behaviour of the comet and other, yet not discussed effects. However, in a short time scale, this approach is sufficient enough to provide a first result that allows one to get an impression of the gravitational perturbations of smaller bodies.

Thomas

References

[1] Maquet, L. (2015). *The recent dynamical history of comet 67P/Churyumov-Gerasimenko*. Astronomy & Astrophysics. Volume 579. Article number A78. DOI: https://doi.org/10.1051/0004-6361/201425461. arXiv: https://arxiv.org/pdf/1412.1983.pdf

Python Space Science Data Science Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play